Milestone 4: Complete Model Training

Regional Dialect Synthesis Pipeline (Standard Hindi to Haryanvi/Bangru)

1. Project & Team Overview

Project Details

Goal: A two-stage pipeline converting Standard Hindi text into spoken Haryanvi.

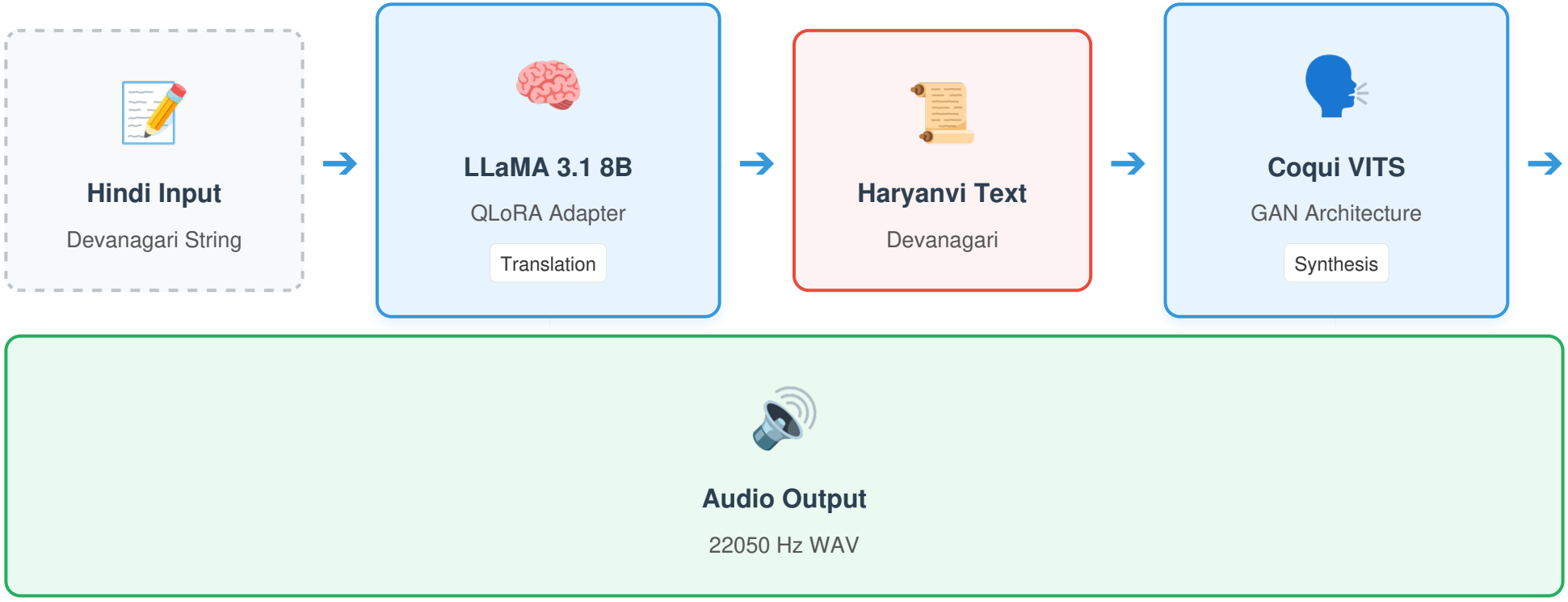
Deadline: 29 March 2026

Team Contributions

Member	Focus Area
Abhishek	End-to-end training of LLaMA (text-to-text model)
Satyam	Hardware management & Training of Coqui-TTS model
Sanket	Documentation & structuring
Fazlur	Authored all code for TTS model pipeline

2. End-to-End Pipeline Architecture

Seamless integration of LLM translation and VITS speech synthesis.



3. Part 1: LLaMA Dataset & Preprocessing

Data Source & Cleaning

- **Dataset:** `Satyam-Srivastava/RDS` (Hugging Face).
- **Format:** Bilingual Hindi/Haryanvi text paired in JSON format.
- **Sanitization:** Merged columns, removed duplicate rows, and dropped null/empty fields.
- **Boundary Filtering:** Applied `TEXT_MIN_WORDS` and `TEXT_MAX_WORDS` to eliminate noise from extremely short or excessively long sequences.

Splits & Sanity Checks

- **Train Split:** Primary optimization target.
- **Validation Split:** Tuning-time early stopping & hyperparameter selection.
- **Test Split:** Final held-out evaluation (strictly separated).
- **Qualitative Holdout:** A fixed sample set for manual side-by-side human review.
- **Truncation Check:** Verified `LLM_MAX_LENGTH` introduces negligible truncation loss across the entire corpus.

4. Part 1: LLaMA Architecture & Strategy

Base Model & QLoRA

- **Base:** `meta-llama/Meta-Llama-3.1-8B-Instruct`.
- **Quantization:** Loaded in 4-bit NF4 (NormalFloat4) with double quantization. NF4 is theoretically optimal for normally-distributed network weights.
- **Adapters Injected:** `q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj`.
- **Compute:** BF16 (with FP16 fallback).

Crucial Design: Assistant-Only Masking

A critical architectural decision was how we handled the loss function:

- System prompt tokens: **Masked (-100)**
- User instruction tokens: **Masked (-100)**
- Padding positions: **Masked (-100)**
- Haryanvi target tokens: **Loss Computed**

Impact: This prevents the model from wasting gradient signals on generating boilerplate conversational text, focusing 100% of its learning capacity on the Hindi → Haryanvi dialect mapping.

5. Part 1: LLaMA Experiments & Metrics

We ran six structured hyperparameter experiments using `paged_adamw_32bit` with Cosine Annealing (Batch size 16 via 4-step accumulation).

Experiment	LR	Epochs	LoRA r/α	Dropout	Eval Loss ↓
lower_lr_longer (Best)	**1e-4**	**4**	16 / 32	0.05	**0.003241**
higher_dropout_reg	2e-4	3	16 / 32	0.15	0.003432
longer_context_256	2e-4	3	16 / 32	0.05	0.003891
stronger_grad_clip	2e-4	3	16 / 32	0.05	0.004164
higher_rank	2e-4	3	**32** / **64**	0.10	0.004269
baseline	2e-4	3	16 / 32	0.05	0.004359

Evaluation Pipeline Bug: Exact match (50.42%), BLEU, and chrF metrics returned mathematically identical scores across all 6 runs. This exposed a critical caching/adapter-loading bug in the inference script. Until resolved, final model selection is based strictly on `eval_loss`.

6. Part 1: LLaMA Key Findings & Error Analysis

What Worked

- **LR & Duration:** The `lower_lr_longer` config achieved a **25.7% reduction** in eval loss over baseline, proving the model benefits from slower, extended passes.
- **Adapter Sizing:** Higher LoRA rank (r=32) performed *worse* than baseline (r=16). The adapter capacity at r=16 is already well-matched to our dataset size.
- **chrF Metric:** We rely on `corpus_chrf` as our primary generative metric. Character-level F-scores are much more robust to Haryanvi's morphological variations than strict BLEU.

Error Analysis & Bottlenecks

Based on worst-case validation exports (sorted by low sentence-level chrF), failure modes are concentrated in:

- **Proper nouns/Named entities** with no aligned equivalent in training.
- **Idiomatic expressions** requiring non-compositional translation.
- **Complex compound sentences** requiring major structural reordering.

Solution: The parallel corpus size is the primary ceiling. Future work requires data augmentation via back-translation and RAG/few-shot prompting with dialect glossaries.

7. Part 2: VITS Dataset & Preprocessing

Raw Data & Transliteration

- **Source:** `ankitdhiman/haryanvi-tts`. Contains mixed-script transcriptions.
- **Bengali to Devanagari:** Processed using `indic-transliteration` library.
- **Urdu/Arabic to Devanagari:** Processed via a custom hardcoded mapping map (libraries failed on Python 3.12).
- **Script Uniformity:** Enforced **Unicode NFC normalization**. Without this, the tokenizer silently drops decomposed characters, ruining attention alignments.

Filtering & Output

- **Boundary Filtering:** Restricted utterances to **3–25 words**. Excludes clips too short for prosody or too long for attention stability.
- **Audio Formatting:** Resampled to **22050 Hz mono 16-bit PCM WAV**.
- **Silence Trimming:** Trimmed at 45 dB threshold.
- **Final Yield:** 2764 rows (~1.5 hours of audio). Split into 2714 train / 50 eval. Exported to standard LJSpeech format.

8. Part 2: VITS Architecture & Warm-Start

Model Architecture (~83M Params)

- **Framework:** Conditional VAE with Adversarial Learning (End-to-End GAN, no separate vocoder).
- **Tokenizer Strategy:** 173-character vocabulary. **No phonemizer used.** Bypassing `espeak-ng`'s Hindi locale prevented systematic mispronunciations of Haryanvi constructs (e.g., *कोनी, म्हारा, सै*).

The Pre-Trained Transfer Advantage

We warm-started training from `SYSPIN/vits_Hindi_Female`. Because Hindi and Haryanvi share ~85-90% phonology, weight transfer was highly successful:

- Waveform Decoder: **100% Loaded** ✅
- Normalizing Flow: **100% Loaded** ✅
- Duration Predictor: **100% Loaded** ✅
- Posterior Encoder: **100% Loaded** ✅

Result: The model bypassed the GAN "babbling/noise" phase entirely, producing recognizable phonemes from Epoch 1.

9. Part 2: VITS Training Configuration & Stack

Tech Stack & Discarded Tools

- **Selected Framework:** `coqui-tts` (idiap fork). Maintained, supports Python 3.12, includes GAN trainer.
- **Discarded: HuggingFace Trainer API** (Cannot handle 2 optimizers and 5 alternating GAN losses).
- **Discarded: AI4Bharat Forks** (Failed to build on Python 3.12, missing modern dependencies).
- **Discarded: Original VITS Format** (Key mismatches; only 34% of decoder loaded).

Hyperparameters (T4 GPU)

- **Hardware:** Google Colab T4 (16 GB VRAM).
- **Precision:** Mixed FP16 (allowed Batch Size 16 without OOM).
- **Learning Rate:** `1e-5` for *both* Generator and Discriminator. Equal LR kept adversarial balance stable.
- **Epochs:** 300 total (~50,700 steps).
- **Early Stopping:** `save_best_after=500` prevents overwriting checkpoints during early GAN volatility.

10. Part 2: VITS Training Progression (300 Epochs)

Training converged beautifully over 300 epochs. Selected Best Checkpoint: `best_model_48503.pth`.

Metric	Initial	Best (Min)	Epoch
loss_mel	~32.0	**13.36**	287
loss_dur	~2.12	1.64	297
loss_kl	~3.70	1.29	269
loss_disc	~2.82	2.55	7

Observed Limitation: Nukta Character Gap
Characters like `ख़` and `ड़` were missing from the training vocabulary and were silently discarded during inference. Requires an audit of the character config against the real-world inference distribution.

Stability Signals:

- `loss_disc` stabilized at ~2.75, confirming **no mode collapse**.
- Long-form inference is stable: 5-minute story outputs (`test_haryanvi_output5min.wav`) synthesized without attention collapse.

11. Roadmap & Next Steps

1. Compute Scale-Up & Extended Training

- `loss_mel` is still declining at Epoch 300. Pushing to **500 epochs** is projected to drop Mel loss to the 8–10 range (a jump from "robotic" to "natural phrasing").
- Requires transitioning from T4 (~42 hours) to **A100 hardware** (~5 hours).

2. Dataset Expansion

- The current ~1.5h dataset is the minimum viable threshold.
- Scale to **5–10 hours** using SNR filtering (< 30 dB excluded) and speed perturbation (±10%) to increase volume without new recordings.

3. Rigorous Formal Evaluation

- Transition from subjective listening to reproducible benchmarks:
- **MOS** (Mean Opinion Score) for human perceptual rating.
- **WER** (Word Error Rate) via Devanagari ASR systems.
- **MCD** (Mel Cepstral Distortion) calculations.

4. End-to-End Stress Testing

- Run the full combined pipeline (LLaMA output → VITS input) on a diverse holdout set to analyze how text-translation artifacts affect TTS prosody.

12. Generated Audio Outputs (Demos)

The pipeline successfully generated 11 distinct `.wav` samples using the final `best_model_48503.pth` checkpoint. These demonstrate various dialectal stresses and long-form attention stability.

Short-Form & Conversational

- **test_haryanvi_output.wav:** Complex sentence (weather, farming).
- **test_haryanvi_output2.wav:** Short idiomatic expressions.
- **test_haryanvi_output4.wav – 11.wav:** 8 diverse sentences covering village life, relationships, and complaints.

Long-Form Stress Tests

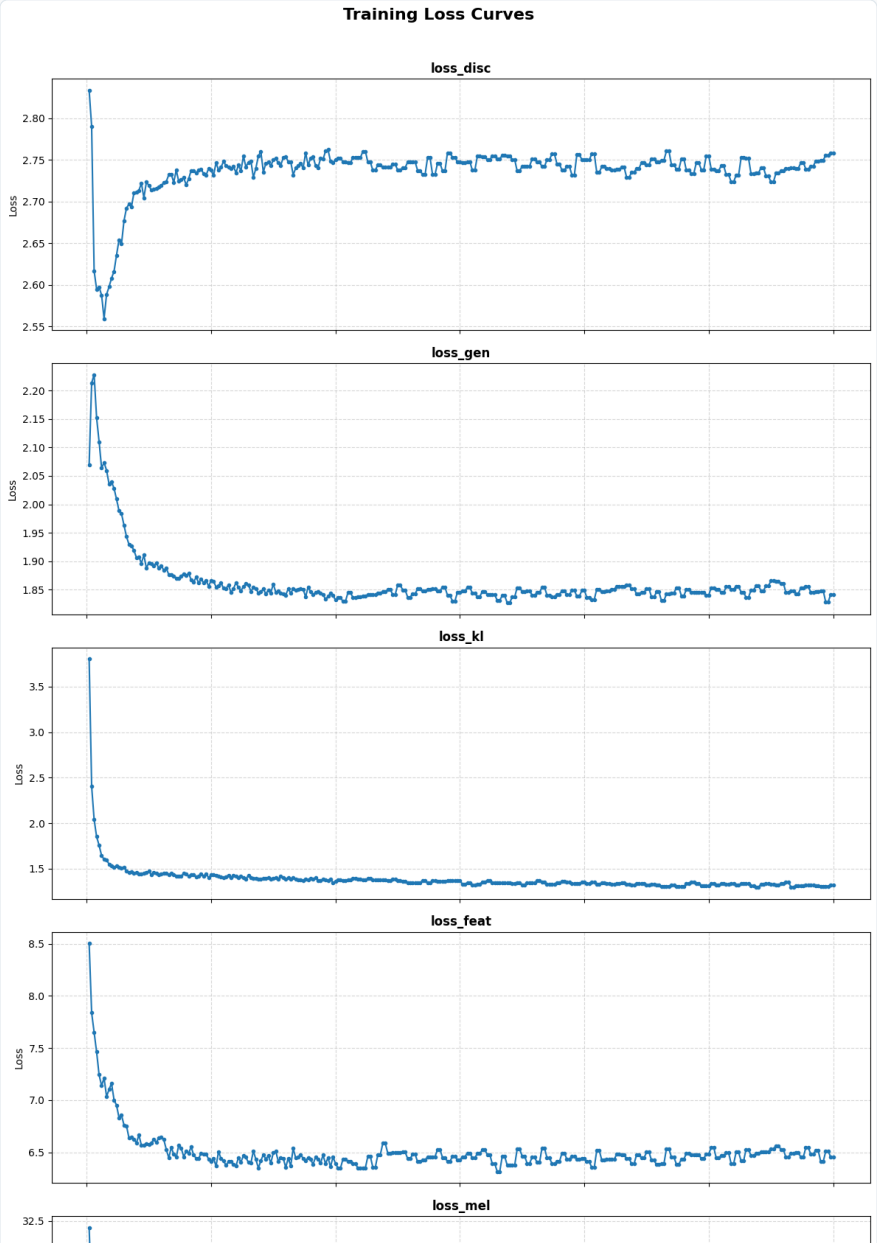
These test the VITS duration predictor and attention maps over multi-paragraph inputs:

- **test_haryanvi_outputfinal.wav:** Short narrative (Ramfal and the old man).
- **test_haryanvi_output5min.wav:** ~5-minute story (Surta and the injured deer).
- **test_haryanvi_output5mintoughtest.wav:** Dialogue-heavy, toughest prosody test.

13. Convergence: 50 Epochs vs 300 Epochs

Comparing our early training phase to our final convergence demonstrates why extended GAN training was critical for this low-resource dialect.

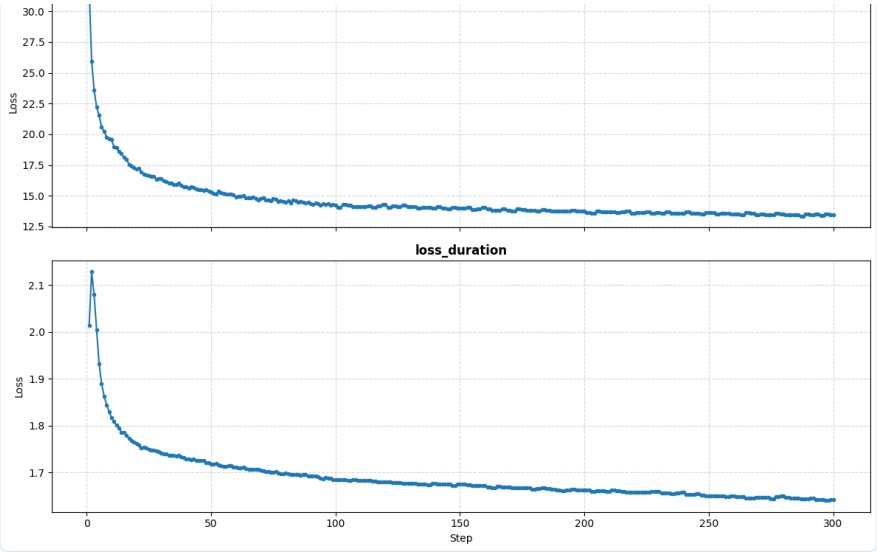
Phase 1: Early Instability (50 Epochs)



Phase 2: Final Convergence (300 Epochs)

Final 300 Epoch Run

Stable adversarial balance. Mel loss drops to 13.36.



Initial GAN volatility. Mel loss plateauing around ~15.

14. Appendix: Project Files & Artifacts

All source code, datasets, adapter weights, and generated audio files are available at the links below:

Code & Data Sources

- 🤗 **Hugging Face Models & Datasets**

  Base bilingual RDS dataset used for LLaMA tuning.

- **Source Code & Jupyter Notebooks**

Model Weights & Outputs

- 📁 **Part 1: LLaMA Translation Outputs**

  LoRA adapters, hyperparams, and validation CSVs.

- 📁 **Part 2: VITS Synthesis Checkpoints**

  Includes best_model and HF warm-start weights.

- 🔊 **Generated Audio Samples**

  All 11 synthesized `.wav` files for quality review.